

Lab 5. Listas, tuplas, conjuntos y diccionarios

Listas, tuplas, conjuntos y diccionarios

Resuelve todos los esta página comprobará que los ejercicios se han realizado correctamente y no permitirá entregas con errores sintácticos o con demasiados fallos de funcionamiento.

1. Manipulación de secuencias

Implementar una función `splitN(L,n)` que divide la secuencia `L`, que recibe como argumento, en tuplas de `n` elementos consecutivos. La función debe funcionar con cualquier tipo de secuencia, por ejemplo, con rangos. El valor devuelto puede ser cualquier iterable, incluyendo generadores. El tipo de secuencia devuelto puede ser cualquier iterable, incluso un generador.

2. Matriz de adyacencia

Implementar una función `matriz_adj(L)` que devuelve la *matriz de adyacencia* correspondiente al grafo descrito por el parámetro `L`. Un grafo no es más que un conjunto de nodos conectado mediante un conjunto de arcos. Los nodos se representan como números enteros, los arcos se representan como tuplas con nodo origen y destino. El parámetro `L` simplemente contiene una lista de los arcos.

Un grafo es *dirigido* cuando el arco `(orig, dest)` implica que el nodo `orig` está conectado (es adyacente) al nodo `dest` pero el nodo `dest` no se considera conectado (adyacente) al nodo `orig`. En este ejercicio se asume que el grafo es dirigido.

La *matriz de adyacencia* para un grafo con nodos $n_0, n_1, n_2, \dots, n_{N-1}$ es una matriz cuadrada $N \times N$ tal que cada elemento es $a_{ij} = 1$ si existe el arco (n_i, n_j) en el grafo, y $a_{ij} = 0$ en caso contrario.

La matriz de adyacencia devuelta debe implementarse como una tupla de `N` tuplas de `N` elementos, de manera que el elemento a_{ij} corresponda al elemento `A[j][i]` de la matriz. Los nodos $n_0, n_1, n_2, \dots, n_{N-1}$ corresponden a los enteros ordenados en orden numérico.

Ejemplo.

```
>>> matriz_adj([(2,3), (2,4), (4,5), (5,2)])
((0,0,0,1), (1,0,0,0), (1,0,0,0), (0,0,1,0))
```

3. Árboles binarios

Implementar una función `arbol_binario(L)` que devuelve el *árbol binario* correspondiente a la lista de números que se indica en el argumento `L`.

Un árbol binario es una tupla de tres elementos o bien `None` si el árbol está vacío. El primer elemento es un valor numérico `n`. El segundo elemento es un árbol binario con todos los números inferiores a `n`. El tercer elemento es un árbol con todos los números superiores a `n`.

Ejemplo.

```
>>> arbol_binario([3, 8, 1, 13, 5, 9])
(3, (1, None, None), (8, (5, None, None), (13, (9, None, None), None)))
```

4. Búsqueda en árbol

Implementar una función `buscar(arbol, valor)` que busca el valor indicado en el segundo argumento dentro del *árbol binario* pasado como primer argumento. Debe devolver `True` si el valor se encuentra en el árbol y `False` en caso contrario.

5. Árbol a conjunto

Implementar una función `arbol_a_conjunto(A)` que devuelve un conjunto con los elementos del árbol `A`.

6. Mezcla de diccionarios

Implementar una función `mezcla(A,B)` que devuelve un diccionario que mezcla los elementos de **A** y **B**. La función no debe alterar los diccionarios que se pasan como argumento. En caso de que una clave exista en los dos diccionarios, la clave del diccionario devuelto deberá estar asociada a una tupla con los dos valores.

7. Búsqueda de ciclos

Implementar una función `hay_ciclo(G)` que devuelve **True** si el grafo **G** contiene al menos un ciclo y **False** en caso contrario. El grafo se modela como una lista de arcos, como en el primer ejercicio.

Un ciclo es una secuencia de arcos que tiene el mismo origen y destino.

8. Conjugación

Implementar una función `presente_indicativo(verbo)` que devuelve una lista con la conjugación en *presente de indicativo* del verbo que se pasa como argumento. Se asumen verbos regulares.

Ejemplo.

```
>>> presente_indicativo('cantar')
['canto', 'cantas', 'canta', 'cantamos', 'cantáis', 'cantan']
>>> presente_indicativo('temer')
['temo', 'temes', 'teme', 'tememos', 'teméis', 'temen']
>>> presente_indicativo('pulir')
['pulo', 'pules', 'pule', 'pulimos', 'pulís', 'pulen']
```

9. Cuartiles

Implementar una función `cuartiles(L)` que devuelve los cuartiles de 25% (Q1), 50% (Q2), 75% (Q3) y el máximo (Q4) de la secuencia de números que se proporciona como argumento.

Consulta [esta página](#) para una descripción del cálculo a realizar.

10. Cambio de notación de RPN a notación algebraica

Implementar una función `rpn_to_algebraic(s)` que recibe una cadena con una expresión algebraica en notación polaca inversa (RPN) y devuelve una cadena con la misma expresión en notación algebraica.

La notación polaca inversa es utilizada en muchas calculadoras científicas porque hace innecesario el uso de paréntesis. Consiste en escribir primero los argumentos y luego el operador. Por ejemplo:

Ejemplo.

```
>>> rpn_to_algebraic('12 3 - 2 5 * +')
'((12 - 3) + (2 * 5))'
```